```
case class Await[I,O](
    recv: Option[I] => Process[I,O])
  extends Process[I,O]

case class Halt[I,O]() extends Process[I,O]
```

A `Process[I,O]` can be used to transform a stream containing `I` values to a stream of `O` values. But `Process[I,O]` isn't a typical function `Stream[I] => Stream[O]`, which could consume the input stream and construct the output stream. Instead, we have a state machine that must be driven forward with a *driver*, a function that simultaneously consumes both our `Process` and the input stream. A `Process` can be in one of three states, each of which signals something to the driver:

- `Emit(head,tail)` indicates to the driver that the `head` value should be emitted to the output stream, and the machine should then be transitioned to the `tail` state.
- `Await(recv)` requests a value from the input stream. The driver should pass the next available value to the `recv` function, or `None` if the input has no more elements.
- `Halt` indicates to the driver that no more elements should be read from the input or emitted to the output.

Let's look at a sample driver that will actually interpret these requests. Here's one that transforms a `Stream`. We can implement this as a method on `Process`:

```
def apply(s: Stream[I]): Stream[O] = this match {
  case Halt() => Stream()
  case Await(recv) => s match {
    case h #:: t => recv(Some(h))(t)
    case xs => recv(None)(xs)        <-- Stream is empty.
  }
  case Emit(h,t) => h #:: t(s)
}
```

Thus, given `p: Process[I,O]` and `in: Stream[I]`, the expression `p(in)` produces a `Stream[O]`. What's interesting is that `Process` is agnostic to how it's fed input. We've written a driver that feeds a `Process` from a `Stream`, but we could also write a driver that feeds a `Process` from a file. We'll get to writing such a driver a bit later, but first let's look at how we can construct a `Process`, and try things out in the REPL.

15.2.1 Creating processes

We can convert any function `f: I => O` to a `Process[I,O]`. We just `Await` and then `Emit` the value received, transformed by `f`:

```
def liftOne[I,O](f: I => O): Process[I,O] =
  Await {
    case Some(i) => Emit(f(i))
    case None => Halt()
  }
```

Let's try this out in the REPL:

```
scala> val p = liftOne((x: Int) => x * 2)
p: Process[Int,Int] = Await(<function1>)

scala> val xs = p(Stream(1,2,3)).toList
xs: List[Int] = List(2)
```

As we can see, this `Process` just waits for one element, emits it, and then stops. To transform a whole stream with a function, we do this repeatedly in a loop, alternating between awaiting and emitting. We can write a combinator for this, `repeat`, as a method on `Process`:

```
def repeat: Process[I,O] = {
  def go(p: Process[I,O]): Process[I,O] = p match {
    case Halt() => go(this)
    case Await(recv) => Await {
      case None => recv(None)
      case i => go(recv(i))
    }
    case Emit(h, t) => Emit(h, go(t))
  }
  go(this)
}
```

Restart the process if it halts on its own.

Don't repeat if terminated from source.

This combinator replaces the `Halt` constructor of the `Process` with a recursive step, repeating the same process forever.

We can now lift any function to a `Process` that maps over a `Stream`:

```
def lift[I,O](f: I => O): Process[I,O] = liftOne(f).repeat
```

Since the `repeat` combinator recurses forever and `Emit` is strict in its arguments, we have to be careful not to use it with a `Process` that never waits! For example, we can't just say `Emit(1).repeat` to get an infinite stream that keeps emitting `1`. Remember, `Process` is a stream *transducer*, so if we want to do something like that, we need to transduce one infinite stream to another:

```
scala> val units = Stream.continually(())
units: scala.collection.immutable.Stream[Unit] = Stream((), ?)

scala> val ones = lift((_:Unit) => 1)(units)
ones: Stream[Int] = Stream(1, ?)
```

We can do more than map the elements of a stream from one type to another—we can also insert or remove elements. Here's a `Process` that filters out elements that don't match the predicate `p`:

```
def filter[I](p: I => Boolean): Process[I,I] =
  Await[I,I] {
    case Some(i) if p(i) => emit(i)
    case _ => Halt()
  }.repeat
```

We simply await some input and, if it matches the predicate, emit it to the output. The call to `repeat` makes sure that the `Process` keeps going until the input stream is exhausted. Let's see how this plays out in the REPL:

```
scala> val even = filter((x: Int) => x % 2 == 0)
even: Process[Int,Int] = Await(<function1>)

scala> val evens = even(Stream(1,2,3,4)).toList
evens: List[Int] = List(2, 4)
```

Let's look at another example of a `Process`, `sum`, which keeps emitting a running total of the values seen so far:

```
def sum: Process[Double,Double] = {
  def go(acc: Double): Process[Double,Double] =
    Await {
      case Some(d) => Emit(d+acc, go(d+acc))
      case None => Halt()
    }
  go(0.0)
}
```

This kind of definition follows a common pattern in defining a `Process`—we use an inner function that tracks the current state, which in this case is the total so far.

Here's an example of using `sum` in the REPL:

```
scala> val s = sum(Stream(1.0, 2.0, 3.0, 4.0)).toList
s: List[Double] = List(1.0, 3.0, 6.0, 10.0)
```

Let's write some more `Process` combinators to help you get accustomed to this style of programming. Try to work through implementations of at least some of these exercises until you get the hang of it.

EXERCISE 15.1

Implement `take`, which halts the `Process` after it encounters the given number of elements, and `drop`, which ignores the given number of arguments and then emits the rest. Also implement `takeWhile` and `dropWhile`, that take and drop elements as long as the given predicate remains true.

```
def take[I](n: Int): Process[I,I]

def drop[I](n: Int): Process[I,I]

def takeWhile[I](f: I => Boolean): Process[I,I]

def dropWhile[I](f: I => Boolean): Process[I,I]
```